Did Anyone Break the PM2.5 Standard?

A hands-on look at one year of real EPA AQS monitoring data · MARAMA Python Tutorial

⚠️ Instructor / solutions copy

Every exercise is written out and the notebook has been executed. The student copy is `test_file_v3.ipynb`.

🎯 What this notebook is

Three blocks. Each starts from a clean slate, so you can **join any block even if the one before did not go to plan**.

- **Block A: Get a clean table.** A messy raw file becomes usable data.
- **Block B: Plot it.** One line per site, then zoom in on a pollution episode.
- **Block C: Ask the regulatory question.** Hours or days? Daily or annual?

You write the code, one small piece at a time.

✍️ How it works

- **Shift + Enter** runs a cell. **Do not use "Run All."** It stops at the first cell you have not written.
- **Write it** cells are empty on purpose. Early ones give you most of the line; later ones give you the goal. Cells with a finished example are labeled **worked example**: run them and read them.
- `check()` looks at **what your code produced**. Many different lines can be right. A wrong result gets a nudge, never the answer.
- Where you see **Predict**, lock in an answer *before* you run the code.
- **Stuck for more than 3 minutes?** Open the hint, then run the **checkpoint** cell at the top of the next block, and put ❓ in the chat so a helper knows. **Finished a block?** Put ✅ in the chat.

Step 0 Set up

Imports, the data file, and one helper file

```
import pandas as pd                    # data tables ("DataFrames"); think Excel, but
programmable
import numpy as np                     # numbers and math
import matplotlib.pyplot as plt        # plotting
import matplotlib.dates as mdates      # tools for dates on plot axes

from pathlib import Path               # a safe way to talk about file locations
```

```
from marama_helpers import check, lock, reveal, scorecard, load_clean_table   # checking helpers
(a separate file)
```

Here we define the path to the data file as a string (str). A string is a piece of text between quotes.

```
# Look for the data file in the places it is likely to be.
# (On Colab: upload it, and marama_helpers.py, with the folder icon on the left. They land in /
content.)
file_name = "AMP501_2420560-0.txt"
candidates = [
    Path("data") / file_name,            # running from the repo's main folder
    Path("..") / "data" / file_name,     # running from a sub-folder such as test/
    Path("/content") / file_name,        # Google Colab upload location
    Path("/mnt/workspace/projects/tutorials/marama_2026/data") / file_name,   # the instructor's
machine
]

# next() takes the FIRST match; None is what we get if nothing was found
path_to_file = next((p for p in candidates if p.is_file()), None)

# If we did not find it, stop here and say so clearly (fail early, fail loudly)
if path_to_file is None:
    raise FileNotFoundError(f"Could not find {file_name}. Upload it, or check the folder.")

print("Found it:", path_to_file.resolve())
```

```
Found it: /mnt/workspace/projects/tutorials/marama_2026/data/AMP501_2420560-0.txt
```

Step ★ First, the answer we are chasing

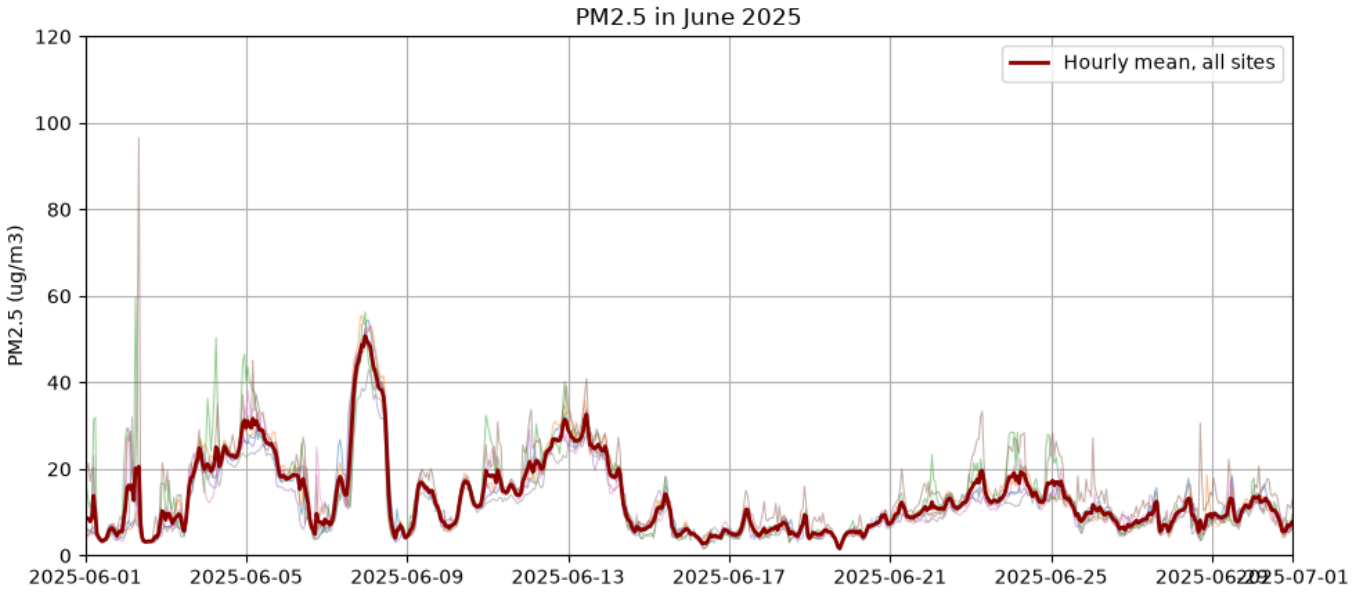
Run this cell. You do not need to understand it yet. By the end you will have built it.

This is one year of hourly PM2.5 from eight monitors. Look at early June.

```
# WORKED EXAMPLE: run it and look. (load_clean_table builds the table you will make yourself in
Block A.)
demo = load_clean_table(path_to_file)

fig, ax = plt.subplots(figsize=(10, 4.5), layout="tight")
for site in demo["Site ID"].unique():
    s = demo[demo["Site ID"] == site]
    ax.plot(s["Datetime"], s["Sample Value"], linewidth=0.7, alpha=0.5)
hourly = demo.groupby("Datetime")["Sample Value"].mean()
ax.plot(hourly.index, hourly.values, color="darkred", linewidth=2, label="Hourly mean, all
sites")
ax.set_xlim(pd.Timestamp("2025-06-01"), pd.Timestamp("2025-07-01"))
ax.set_ylim(0, 120)
ax.set_ylabel("PM2.5 (ug/m3)")
ax.set_title("PM2.5 in June 2025")
ax.grid("both")
ax.legend()
plt.show()
```

```
📦 Loaded the clean table: 60,264 rows. You can continue from here.
```



> ✍️ **Predict (this one is resolved at the end of Block C)**
>
> The 24-hour PM2.5 standard compares a *daily average* to a limit. Looking at the spikes above: **did any site have a day that broke the 24-hour standard?** Type `"yes"` or `"no"`.

> 💬📊 **Poll**
>
> Vote: yes or no? *(Virtual: answer in the poll window. In the room: raise a hand.)*

```
my_prediction = "yes"
lock("opener", my_prediction)
```

```
🔒 Locked in: 'yes'. Now do the work and see what actually happens.
```

Let's go back to the raw file and find out.

Step A Block A: Get a clean table

From a messy raw file to data you can trust

Before we ask pandas to open it, let's read the first few lines as plain text.

```
# Open the file as plain text and print the first 3 lines (cut at 200 characters so they fit)
with open(path_to_file) as f:
    for i in range(3):
        print(f"line {i}:", f.readline()[:200])
```

```
line 0: # RD|Action Code|State Code|County Code|Site ID|Parameter|POC|Sample Duration|Unit|Method|
Date|Start Time|Sample Value|Null Data Code|Sampling Frequency|Monitor Protocol (MP) ID|Qualifier
- 1|Qualifie
line 1: # RC|Action Code|State Code|County Code|Site ID|Parameter|POC|Unit|Method|Year|Period|
Number of Samples|Composite Type|Sample Value|Monitor Protocol (MP) ID|Qualifier - 1|Qualifier -
2|Qualifier - 3|Q
```

```
line 2: RD|I|42|003|0002|88101|2|1|105|636|20250101|00:00|0.8||||6||||||||||
```

> ✍️ **Look and answer**
>
> **What single character separates the columns?** And what is unusual about lines 0 and 1?

```
separator_i_see = "|"
whats_odd_about_lines_0_and_1 = "two header lines, both starting with #"
```

```
check("sep", separator_i_see)
print("Your note about lines 0 and 1:", whats_odd_about_lines_0_and_1)
```

```
✅ Correct.
Your note about lines 0 and 1: two header lines, both starting with #
```

First we try to naively import the file using `pandas.read_csv()`. It shouldn't work exactly as you want it.

> ✍️ **Predict**
>
> By default, `read_csv()` splits columns at **commas**. What will pandas do with this file?
> **A**: find one column · **B**: find about 28 columns · **C**: stop with an error

> 💬📊 **Poll**
>
> Vote: A, B or C? *(Virtual: answer in the poll window. In the room: raise a hand.)*

```
my_prediction = "A"
lock("naive", my_prediction)
```

```
🔒 Locked in: 'A'. Now do the work and see what actually happens.
```

```
# WORKED EXAMPLE: the naive read
df = pd.read_csv(path_to_file)
print(df.shape[1], "column(s)")
reveal("naive", {1: "A", 28: "B"}.get(df.shape[1], "C"))
```

```
1 column(s)
   Your prediction: 'A'     What happened: 'A'
   You called it.
```

.read_csv() can import more delimiter types than ','. Tell it which one this file uses.

> ✍️ **Write it**
>
> Write the read again, this time telling pandas which character separates the columns. The argument is `sep=`.

```
df = pd.read_csv(path_to_file, sep="|")

check("sep_read", df.shape[1])
```

```
✅ Correct.
```

```
/tmp/ipykernel_3020013/822255439.py:1: DtypeWarning: Columns (2: State Code, 3: County Code, 4:
Site ID, 5: Parameter, 6: POC, 7: Sample Duration, 8: Unit, 9: Method, 10: Date, 12: Sample Value,
14: Sampling Frequency, 15: Monitor Protocol (MP) ID, 16: Qualifier - 1, 17: Qualifier - 2, 18:
Qualifier - 3, 19: Qualifier - 4, 20: Qualifier - 5, 21: Qualifier - 6, 22: Qualifier - 7, 23:
Qualifier - 8, 24: Qualifier - 9, 25: Qualifier - 10, 26: Alternate Method Detectable Limit) have
mixed types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv(path_to_file, sep="|")
```

Stuck? Open a hint (not the answer)

1. The pattern is `pd.read_csv(path_to_file, sep="...")`. What goes between the quotes?

⚠️ That warning, and that last row

The **DtypeWarning** means pandas found conflicting data types in some columns. The second header line looks like data but is text, and pandas guesses a type chunk by chunk. Also look at the last row of the table: it is a footer (`# 60264 records were written`), not data.

💡 Worked example: three fixes in one read

- `header=[0]`: use only the first line as the header.
- `low_memory=False`: read each whole column before guessing its type, so the warning goes away.
- `[:-1]`: a **slice**. `df[a:b]` keeps rows from a up to, not including, b. Negative numbers count from the end, so `[:-1]` keeps everything except the last row.

```
# WORKED EXAMPLE: run it, then read the last rows and the first row
df = pd.read_csv(path_to_file, sep="|", header=[0], low_memory=False)[:-1]
print(df.shape)
df.head(2)
```

```
(60265, 28)
```

	# RD	Action Code	State Code	County Code	Site ID	Parameter	POC	Sample Duration	Unit	Method	...	Qualifier - 3	Qualifier - 4	Qualifier - 5	Qualifier - 6	Qu
0	# RC	Action Code	State Code	County Code	Site ID	Parameter	POC	Unit	Method	Year	...	Qualifier - 4	Qualifier - 5	Qualifier - 6	Qualifier - 7	Qu - 8
1	RD	I	42	003	0002	88101	2	1	105	636	...	NaN	NaN	NaN	NaN	Na

2 rows × 28 columns

💬 Discuss

Look at **row 0** and at the **Dtype** column from `df.info()` (run `df.info()` in a new cell if you like). Sample Value should be numbers. How could a column of numbers end up as text?

Instructor's take (open after you have talked)

Row 0 is the **second header line**, still sitting in the data. One text value turns the whole column into text. That is the same cause as the DtypeWarning. So we need to drop row 0 *and* the footer, and keep Date and Start Time as text on purpose, because we are about to glue them together.

✍️ Write it

Write the read in its **final form**: your separator, `header=[0]` and `low_memory=False`; keep `Date` and `Start Time` as **text** with `dtype=` and a dictionary; and use **one slice** to drop both the top junk row and the footer.

```
dtypes = {
    "Date": str,
    "Start Time": str,
}

df = pd.read_csv(path_to_file, sep="|", header=[0], low_memory=False, dtype=dtypes)[1:-1]

check("clean", (df.iloc[0]["Date"], df.shape))
df[["Date", "Start Time"]].head()
```

✅ Correct.

	Date	Start Time
1	20250101	00:00
2	20250101	01:00
3	20250101	02:00
4	20250101	03:00
5	20250101	04:00

Stuck? Open a hint (not the answer)

1. A dictionary maps names to values: `{"Date": str, "Start Time": str}`. Pass it as `dtype=...`.
2. Row 0 is the junk row at the top, so the slice must start at 1. The footer is the last row, so it must stop at -1.
3. Still stuck? Run the checkpoint at the top of Block B, and put ❓ in the chat.

To do any plotting we need this oriented by time. Right now we have no datetime information, only two strings (Date and Start Time). We can combine them into one string and then convert that to a datetime.

```
# WORKED EXAMPLE: glue the two text columns together, row by row:  "20250101" + "00:00"  ->
"2025010100:00"
glued = df["Date"] + df["Start Time"]
glued.head(3)
```

```
1    2025010100:00
2    2025010101:00
3    2025010102:00
dtype: str
```

✍️ Write it

Write one line that turns `glued` into a datetime column named `Datetime`, using `pd.to_datetime(..., format=...)`. Format codes: `%Y` year, `%m` month, `%d` day, `%H` hour (24-hour), `%M` minute.

```
df["Datetime"] = pd.to_datetime(glued, format="%Y%m%d%H:%M")

check("datetime", str(df["Datetime"].iloc[0]))
```

```
✅ Correct.
```

Stuck? Open a hint (not the answer)

1. The shape of the call is `df["Datetime"] = pd.to_datetime(glued, format="...")`.
2. Write the pattern in the same order as the glued text: year, month, day, hour, a colon, minute.

💬 Discuss

The file does not say what time zone these timestamps are in. Why would that matter if you wanted to line these data up with a **satellite overpass**? (Virtual: type your idea in the chat.)

Instructor's take (open after you have talked)

AQS hourly times are reported in **local standard time**, with no daylight-saving shift (see the AQS format documentation). Satellites report UTC, so you would need each site's offset before matching them.

💡 Missing data is not zero

Some hours have *no reading*. AQS marks them with a **Null Data Code** and leaves the value blank. A blank has to stay **missing** (NaN), or averages get dragged down and nobody knows.

✍️ Write it

Convert `Sample Value` to numbers with `pd.to_numeric()`, turning anything that is not a number into NaN. Then **count the hours with no reading** in `n_missing`. Look up `errors=`, `isna()` and `sum()`.

```
df["Sample Value"] = pd.to_numeric(df["Sample Value"], errors="coerce")
n_missing = int(df["Sample Value"].isna().sum())

print("Hours with no reading:", n_missing)
check("numeric", (str(df["Sample Value"].dtype), n_missing))
```

```
Hours with no reading: 342
✅ Correct.
```

Stuck? Open a hint (not the answer)

1. `errors=` can be `"raise"` (stop with an error), `"ignore"` (leave the text alone) or `"coerce"` (make it NaN). Which fits?
2. `df["Sample Value"].isna()` gives True or False for each row. `.sum()` counts the Trues.

💬 Discuss

The **Null Data Code** column explains the gaps. If you were writing the report, would you drop these hours, fill them in, or report them as missing? Who decides?

Instructor's take (open after you have talked)

There is no single right answer. What matters is that the choice is **made on purpose and written down**. Some AQS codes mean "the monitor ran but the data failed QA," which is very different from "the monitor was off."

✍️⭐ Stretch (optional, for when you finish early)

Which **site** has the most missing hours, and how many? Store the site ID and the count in `worst_site` and `worst_count`.

```
missing_by_site = df.groupby("Site ID")["Sample Value"].apply(lambda s: int(s.isna().sum()))
worst_site, worst_count = missing_by_site.idxmax(), int(missing_by_site.max())

check("stretch_a", (worst_site, worst_count))
```

```
✅ Correct.
```

🎯 End of Block A

You now have a clean table: right columns, no junk rows, real timestamps, real numbers. **Put ✅ in the chat.**

Step B Block B: Plot it

One line per site, then zoom in on an episode

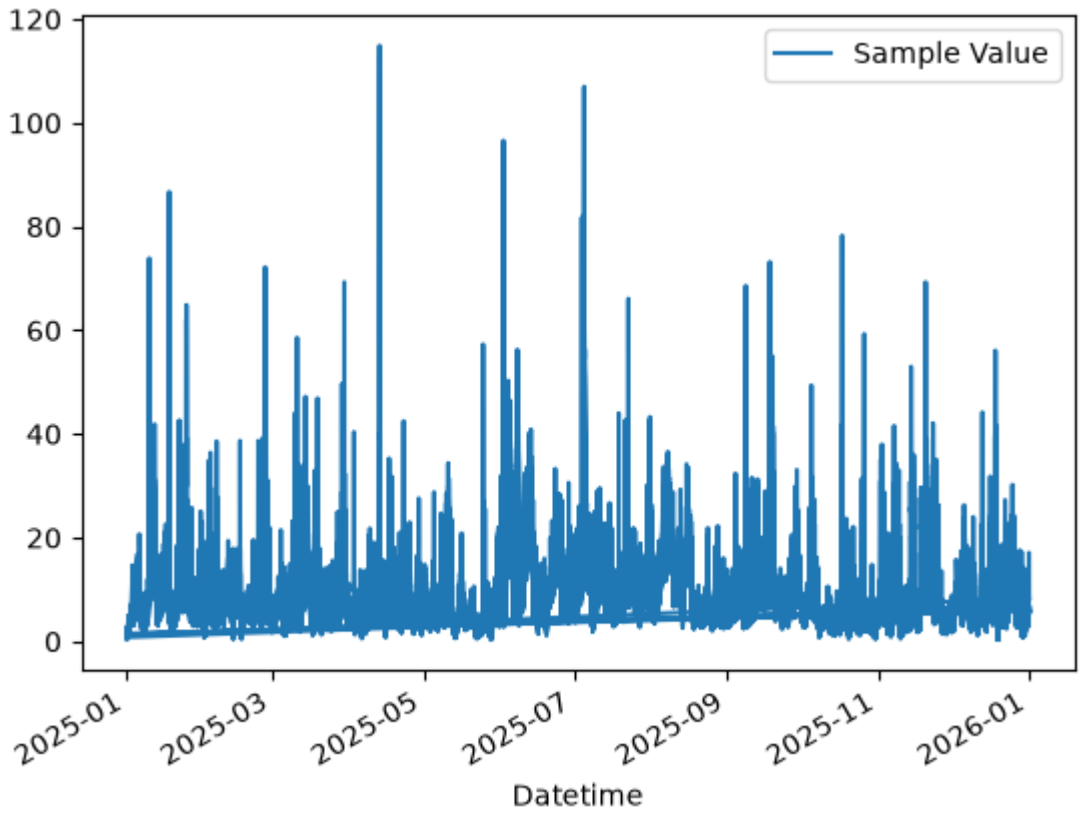
⚠️ Checkpoint: start here if Block A did not work out

Run the next cell to load the clean table. There is no shame in it: this is what it is for. Put ❓ in the chat if you used it, so a helper can catch you up later.

```
df = load_clean_table(path_to_file)    # run this ONLY if your own table from Block A is not working
```

```
📦 Loaded the clean table: 60,264 rows. You can continue from here.
```

```
# WORKED EXAMPLE: the quickest possible plot: x = time, y = the PM2.5 value
df.plot(x="Datetime", y="Sample Value");
```



We got a plot, but wait... are there multiple sites in this file?

> ✍️ **Write it**
>
> Write one line that stores each distinct `Site ID` exactly once in a variable called `sites`. The pandas method's name is a word for "one of a kind."

```
sites = df["Site ID"].unique()

print(sites)
check("sites", sorted(sites))
```

```
<StringArray>
['0002', '0008', '0064', '0067', '1008', '1301', '1376', '3007']
Length: 8, dtype: str
✅ Correct.
```

Stuck? Open a hint (not the answer)

1. Start from the column: `df["Site ID"]`. Then add a method to it. `unique` is one; `drop_duplicates` is another.

We should probably split things up by 'Site ID'. We could make multiple plots (but there are a lot of unique Site IDs), or we could use a legend. Let's use Matplotlib for this part...

> 💡 **Reading a loop out loud**

`for site in sites:` means “take the sites one at a time, call the current one `site`, and run the indented lines.” `df[df["Site ID"] == site]` means “keep only the rows where Site ID equals this site.” Two equals signs: == asks a question, = stores a value.

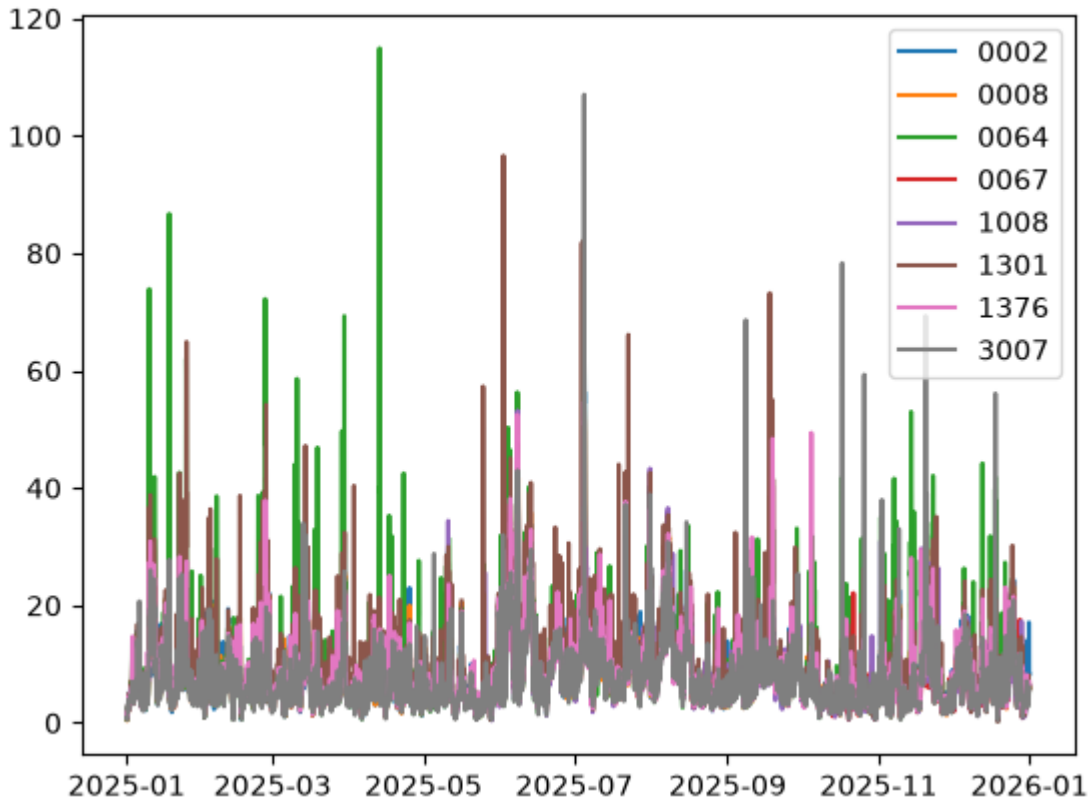
✍️ Write it

Write a **loop** that draws one line per site on the same plot (`Datetime` on x, `Sample Value` on y, the site as the `label`). The loop header is given. You write the body: pick out one site's rows, then call `plt.plot(...)`.

```
plt.figure()
for site in sites:
    df_site = df[df["Site ID"] == site][["Datetime", "Sample Value"]]
    plt.plot(df_site["Datetime"], df_site["Sample Value"], label=site)

plt.legend()
check("loop", len(plt.gca().lines))
plt.show()
```

✅ Correct.



Stuck? Open a hint (not the answer)

1. Inside the loop, make a smaller table: `df_site = df[df["Site ID"] == site]`.
2. Then plot its two columns: `plt.plot(df_site["Datetime"], df_site["Sample Value"], label=site)`.

💬 Discuss

Do the sites move together or apart? Does any site have a gap or a short record? Write down what you see. (Virtual: type it in the chat.)

Instructor's take (open after you have talked)

The sites rise and fall together, so the high periods are **regional**, not one local source. Two lines are visibly shorter. We count them in Block C.

💡 Worked example: a better plot, with a daily mean

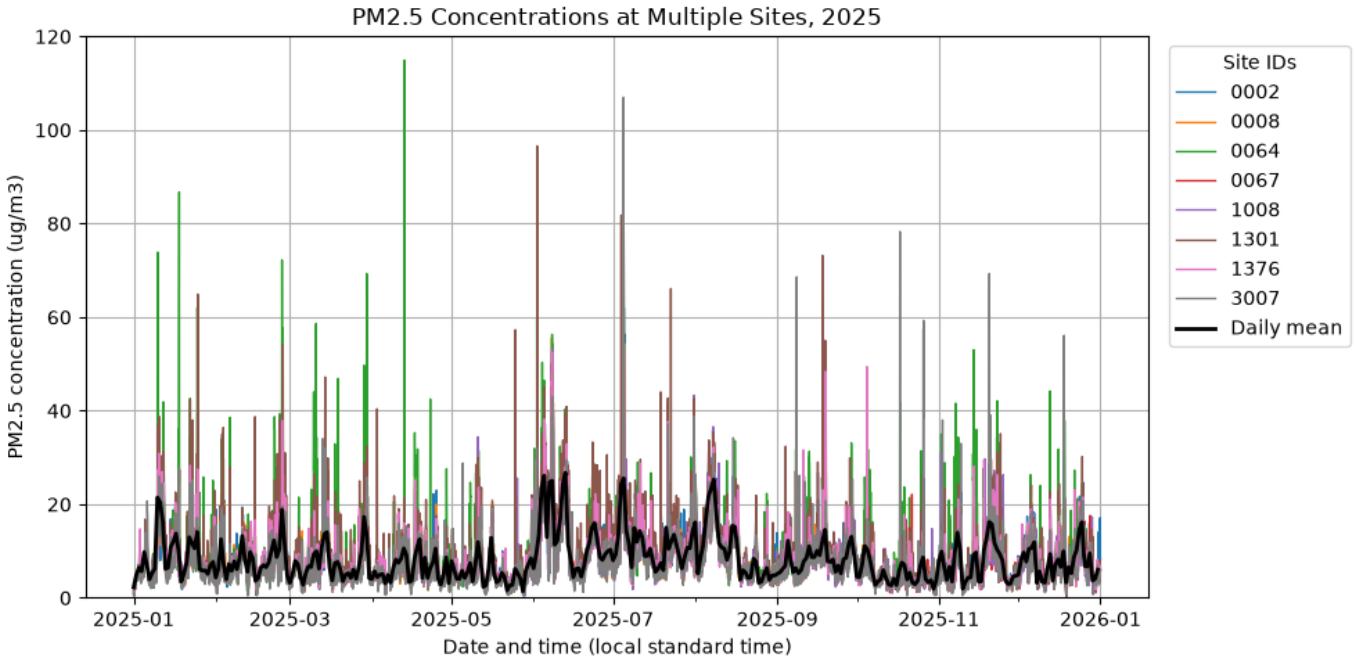
`fig, ax = plt.subplots()` gives you named objects, and `ax.something()` says exactly *which* plot you mean. The **daily mean** groups by the calendar *date* of each timestamp (`.dt.date`), then averages across all sites.

```
# WORKED EXAMPLE: read every line, then run it
fig, ax = plt.subplots(figsize=(10, 5), layout="tight")

for site in sites:
    df_site = df[df["Site ID"] == site][["Datetime", "Sample Value"]]
    ax.plot(df_site["Datetime"], df_site["Sample Value"], label=site, linewidth=1)

# The daily mean across all sites: group by calendar date, then average
daily_mean = df.groupby(df["Datetime"].dt.date, as_index=False)["Sample Value"].mean()
ax.plot(daily_mean["Datetime"], daily_mean["Sample Value"], label="Daily mean", color="black",
linewidth=2)

ax.set_ylabel("PM2.5 concentration (ug/m3)")
ax.set_xlabel("Date and time (local standard time)")
ax.set_title("PM2.5 Concentrations at Multiple Sites, 2025")
ax.set_ylim(0, 120)
ax.grid("both")
ax.xaxis.set_minor_locator(mdates.MonthLocator())
ax.legend(title="Site IDs", bbox_to_anchor=(1.01, 1), loc="upper left")
plt.show()
```



Let's do some analysis on this now. It looks like 2025-06 has some high values for several days across all sites.

> ✍️ **Write it**
>
> Zoom in on June 2025 and add the **hourly mean**. Write three pieces: the **hourly mean** (like the daily mean above, but group by the full `Datetime` column), the **zoom** with `ax.set_xlim(...)`, and a **y-axis label with units and a title**.

```
fig, ax = plt.subplots(figsize=(10, 5), layout="tight")
for site in sites:
    df_site = df[df["Site ID"] == site][["Datetime", "Sample Value"]]
    ax.plot(df_site["Datetime"], df_site["Sample Value"], label=site, linewidth=0.8, alpha=0.6)

ax.plot(daily_mean["Datetime"], daily_mean["Sample Value"], label="Daily mean", color="black",
linewidth=2.5)

hourly_mean = df.groupby("Datetime", as_index=False)["Sample Value"].mean()
ax.plot(hourly_mean["Datetime"], hourly_mean["Sample Value"], label="Hourly mean",
color="darkred", linewidth=2)

ax.set_xlim(pd.Timestamp("2025-06-01"), pd.Timestamp("2025-07-01"))

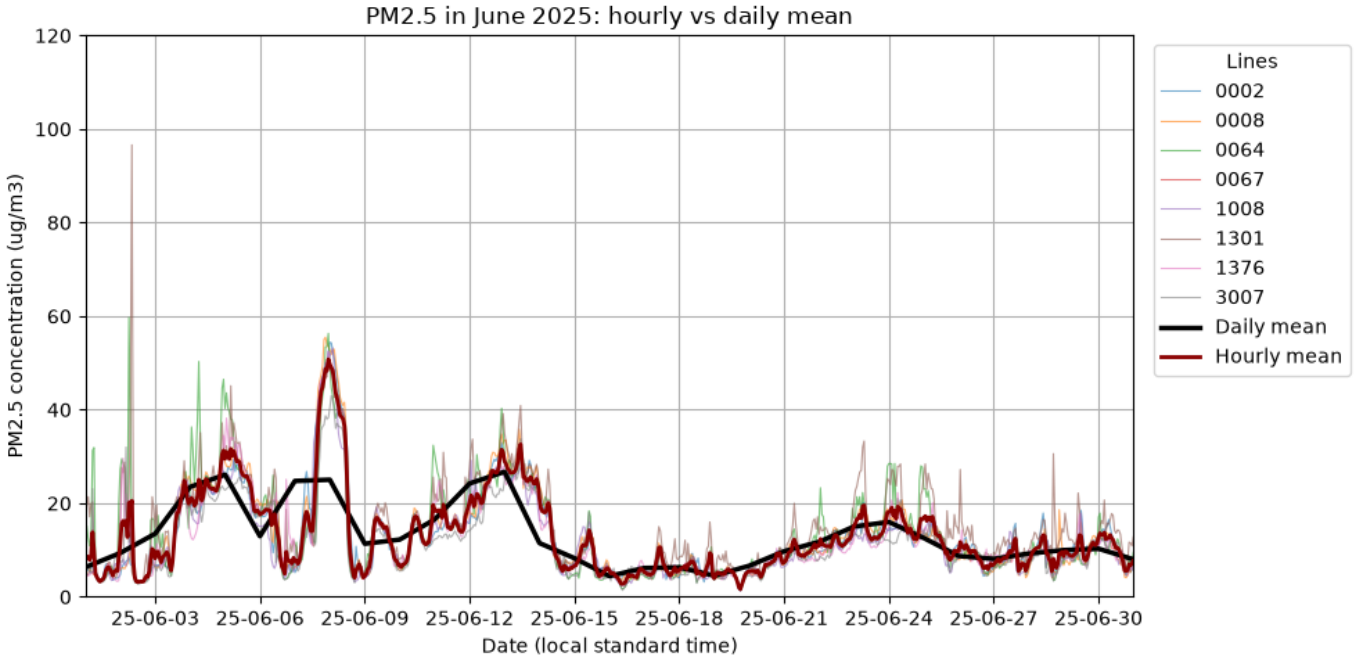
ax.set_ylabel("PM2.5 concentration (ug/m3)")
ax.set_title("PM2.5 in June 2025: hourly vs daily mean")

check("hourly", round(float(hourly_mean["Sample Value"].sum()), 1))
lo, hi = (mdates.num2date(v).strftime("%Y-%m-%d") for v in ax.get_xlim())
check("june", f"{lo}|{hi}")
check("labels", len(ax.get_ylabel()) > 2 and len(ax.get_title()) > 2)

ax.set_ylim(0, 120)
ax.set_xlabel("Date (local standard time)")
```

```
ax.grid("both")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%y-%m-%d"))
ax.xaxis.set_major_locator(mdates.DayLocator(interval=3))
ax.legend(title="Lines", bbox_to_anchor=(1.01, 1), loc="upper left")
plt.show()
```

✅ Correct.
✅ Correct.
✅ Correct.



Stuck? Open a hint (not the answer)

1. The hourly mean looks like the daily mean line above, but it groups by `"Datetime"` itself. Then plot it with `ax.plot(...)`.
2. `ax.set_xlim(pd.Timestamp("..."), pd.Timestamp("..."))`. The end is the first day after June.
3. `ax.set_ylabel("text")` and `ax.set_title("text")`. Still stuck? Run the checkpoint at the top of Block C and put ❓ in the chat.

💬 Discuss

Compare the **red** line and the **black** line. Where do they disagree most? Which would you show a reporter, and which a regulator? (Virtual: chat.)

Instructor's take (open after you have talked)

The hourly mean (red) catches short spikes that the daily mean (black) averages away. Neither is “wrong.” They answer different questions, and the standards are written for a specific averaging time.

✍️ ⭐ Stretch (optional, for when you finish early)

When is the **single highest hourly mean**? Store the timestamp in `peak_time`.

```
peak_time = hourly_mean.loc[hourly_mean["Sample Value"].idxmax(), "Datetime"]

check("stretch_b", peak_time)
```

```
✅ Correct.
```

🎯 End of Block B

Plots that show the episode. **Put ✅ in the chat.**

Step C Block C: Ask the regulatory question

Standards are not set on single hours

⚠️ Checkpoint: start here if Block B did not work out

Run the next cell to load the clean table, then continue. Put ❓ in the chat if you used it.

```
df = load_clean_table(path_to_file)    # run this ONLY if your own table is not working
```

```
📦 Loaded the clean table: 60,264 rows. You can continue from here.
```

🎯 The question

The **24-hour PM2.5 standard** compares a *daily average* to a limit. So: **did any site have a day above it?** We need (1) a daily mean per site and (2) a rule for when a day has enough data to count. A common rule: at least **75% of the hours**.

✍️ Predict

Out of all the site-days, how many pass a 75% completeness rule?
"most" (over 90%) · **"about half"** · **"few"** (under 50%)

```
my_prediction = "about half"
lock("pct", my_prediction)
```

```
🔒 Locked in: 'about half'. Now do the work and see what actually happens.
```

💡 Worked example: a daily table with a completeness rule

Group by **two** columns (site and day). `.agg(...)` with **named results** gives the mean *and* the number of valid readings. `count()` skips missing values, which is why we use it. Study this pattern: you will reuse it twice.

```
# WORKED EXAMPLE: read each line, then run it
df["Day"] = df["Datetime"].dt.date

daily = (
    df.groupby(["Site ID", "Day"])["Sample Value"]
    .agg(daily_mean="mean", valid_hours="count")   # two named results per (site, day)
```

```
    .reset_index()
)

daily["complete"] = daily["valid_hours"] >= 18     # 75% of 24 hours
daily_valid = daily[daily["complete"]]             # keep only the days that count

print(f"{len(daily)} site-days in total; {len(daily_valid)} pass the rule")
share = daily["complete"].mean()
reveal("pct", "most" if share > 0.9 else "about half" if share >= 0.5 else "few")
```

```
2511 site-days in total; 2493 pass the rule
   Your prediction: 'about half'     What happened: 'most'
   Different from your prediction. What did you expect to see, and why?
```

💡 Reading AI-written code

An AI assistant suggested a slightly different rule. It runs without any error. **Read it, then test it.**

```
# An AI assistant wrote this line for the completeness rule:
ai_complete = daily["valid_hours"] > 18
```

✍️ Write it

Write code that counts **how many site-days get a different answer** under the AI's rule (`> 18`) than under ours (`>= 18`). Store the count in `n_different`. Then type, in your own words, whether it matters.

```
n_different = int((ai_complete != daily["complete"]).sum())


my_explanation =
"It matters at the boundary: days with exactly 18 valid hours are dropped by '>' but kept by
'>='."
check("ai_diff", n_different)
print("Your explanation:", my_explanation)
```

```
✅ Correct.
Your explanation: It matters at the boundary: days with exactly 18 valid hours are dropped by '>'
but kept by '>='.
```

Stuck? Open a hint (not the answer)

1. Two True/False columns can be compared with `!=` (not equal). Counting the Trues is `.sum()`.
2. `(ai_complete != daily["complete"]).sum()` is one way. Why would the two rules differ for some days and not others?

💬 Discuss

Is it a big number or a small one? Does that make it unimportant? Who decides which rule is right? (Virtual: chat.)

Instructor's take (open after you have talked)

The rules only differ for days sitting **exactly on the boundary**. A small count can still change a regulatory result, and "at least 75%" means exactly 18 hours should count. A confident-looking one-character difference in generated code is exactly what careful reading catches.

✍️ Predict

Back to the opening question. Did any site-day exceed the 24-hour standard? The notebook will reveal your opening prediction when you have the answer.

💬📊 Poll

Vote again: yes or no? Did the room change its mind? *(Virtual: answer in the poll window. In the room: raise a hand.)*

✍️ Write it

Set `NAAQS_24HR` to the primary 24-hour PM2.5 standard (µg/m³), then write the code that keeps the rows of `daily_valid` with a daily mean **above** it, in `exceed`. Then print: the number of rows, the highest **hourly** value in `df`, and the highest **daily** mean.

```
NAAQS_24HR = 35
exceed = daily_valid[daily_valid["daily_mean"] > NAAQS_24HR]

check("naaqs24", NAAQS_24HR)
check("exceed", len(exceed))
reveal("opener", "yes" if len(exceed) else "no")

print("Site-days above the standard:", len(exceed))
print("Highest HOURLY value in the file :", df["Sample Value"].max(), "µg/m³")
print("Highest valid DAILY mean         :", round(daily_valid["daily_mean"].max(), 1), "µg/m³")
```

```
✅ Correct.
✅ Correct.
   Your prediction: 'yes'     What happened: 'no'
   Different from your prediction. What did you expect to see, and why?
Site-days above the standard: 0
Highest HOURLY value in the file : 114.9 µg/m³
Highest valid DAILY mean         : 33.7 µg/m³
```

Stuck? Open a hint (not the answer)

1. The standard is a whole number between 30 and 40. Look it up in the EPA NAAQS table.
2. `daily_valid[daily_valid["daily_mean"] > NAAQS_24HR]` keeps the rows above the limit.
3. `df["Sample Value"].max()` and `daily_valid["daily_mean"].max()` give the two highest values.

💬 Discuss

Compare the highest **hourly** value, the highest **daily** mean, and the standard. Why can one hour be far above a limit while no *day* is? And which day has the highest daily mean? Is it where you expected?

Instructor's take (open after you have talked)

Averaging time decides the answer. **Always ask what number the standard actually compares.** The single highest daily mean (site 1301) is on **July 4**, not in June, and the highest hourly network mean falls late that night. A holiday evening is a plausible cause, but the data alone do not prove it.

⚠️ A screening number, not a design value

The regulatory statistic for the 24-hour standard is the **98th percentile** of daily means, averaged over **three years**. We have one year, and the pandas percentile method is not the official one. Treat what follows as **screening**.

✍️ Write it

Make a table `site_summary` with one row per site: `valid_days` (count), p98 (the 98th percentile of its daily means), and `max_daily`. Round to one decimal, sorted with the highest p98 first. `quantile()` takes a fraction between 0 and 1. **Reuse the pattern from the daily-table worked example.**

```
site_summary = (
    daily_valid.groupby("Site ID")["daily_mean"]
    .agg(valid_days="count", p98=lambda s: s.quantile(0.98), max_daily="max")
    .round(1)
    .sort_values("p98", ascending=False)
)

check("p98", site_summary["p98"].iloc[0])
site_summary
```

✅ Correct.

	valid_days	p98	max_daily
Site ID			
1301	361	25.4	33.7
0008	360	24.3	28.5
1008	236	24.2	29.2
0064	365	24.0	29.5
0002	365	22.6	28.4
1376	362	21.6	28.3
3007	361	20.5	27.5
0067	83	14.1	15.3

Stuck? Open a hint (not the answer)

1. Group `daily_valid` by `"Site ID"`, pick `"daily_mean"`, and use `.agg(...)` with three named results, as in the worked example.
2. A named result with a custom function: `p98=lambda s: s.quantile(...)`.
3. `.round(1)` then `.sort_values("p98", ascending=False)`.

💬 Discuss

Look at the `valid_days` column. What do you notice about two of the sites? Can you fairly compare them with the others? (Virtual: chat.)

Instructor's take (open after you have talked)

Sites 0067 and 1008 have far fewer valid days (the short records you may have spotted in the line plot). A low number from a site that ran for only part of the year can look great and mean nothing.

✍️ Predict

Will **every** site be below the **annual** standard? Type `"all below"` or `"at least one above"`.

💬📊 Poll

Vote: all below, or at least one above? *(Virtual: answer in the poll window. In the room: raise a hand.)*

```
my_prediction = "all below"
lock("over_yn", my_prediction)
```

```
🔒 Locked in: 'all below'. Now do the work and see what actually happens.
```

✍️ Write it

Set NAAQS_ANNUAL to the primary annual PM2.5 standard (µg/m³). Make a table `annual` with each site's **annual mean** (the average of its valid daily means), rounded to one decimal, plus a True/False column `above_standard`.

```
NAAQS_ANNUAL = 9.0

annual = (
    daily_valid.groupby("Site ID")["daily_mean"]
    .agg(annual_mean="mean", valid_days="count")
    .round(1)
    .sort_values("annual_mean", ascending=False)
)
annual["above_standard"] = annual["annual_mean"] > NAAQS_ANNUAL

check("annual", NAAQS_ANNUAL)
check("annual_top", annual["annual_mean"].max())
reveal("over_yn", "at least one above" if annual["above_standard"].any() else "all below")
annual
```

```
✅ Correct.
✅ Correct.
   Your prediction: 'all below'     What happened: 'at least one above'
   Different from your prediction. What did you expect to see, and why?
```

	annual_mean	valid_days	above_standard
Site ID			
1301	9.5	361	True
0064	8.9	365	False
1008	8.4	236	False

	annual_mean	valid_days	above_standard
Site ID			
0002	8.1	365	False
0008	7.9	360	False
1376	7.8	362	False
3007	7.4	361	False
0067	5.8	83	False

Stuck? Open a hint (not the answer)

1. The annual standard is a single-digit number. Look it up in the EPA NAAQS table.
2. Same pattern again: group `daily_valid` by site, take the mean of `daily_mean`, name the result `annual_mean`.
3. `annual["above_standard"] = annual["annual_mean"] > NAAQS_ANNUAL`.

💬 Discuss

The 24-hour check and the annual check disagree. Which kind of pollution does each standard protect against? Is a result this close to a limit a conclusion, or a reason to look closer?

Instructor's take (open after you have talked)

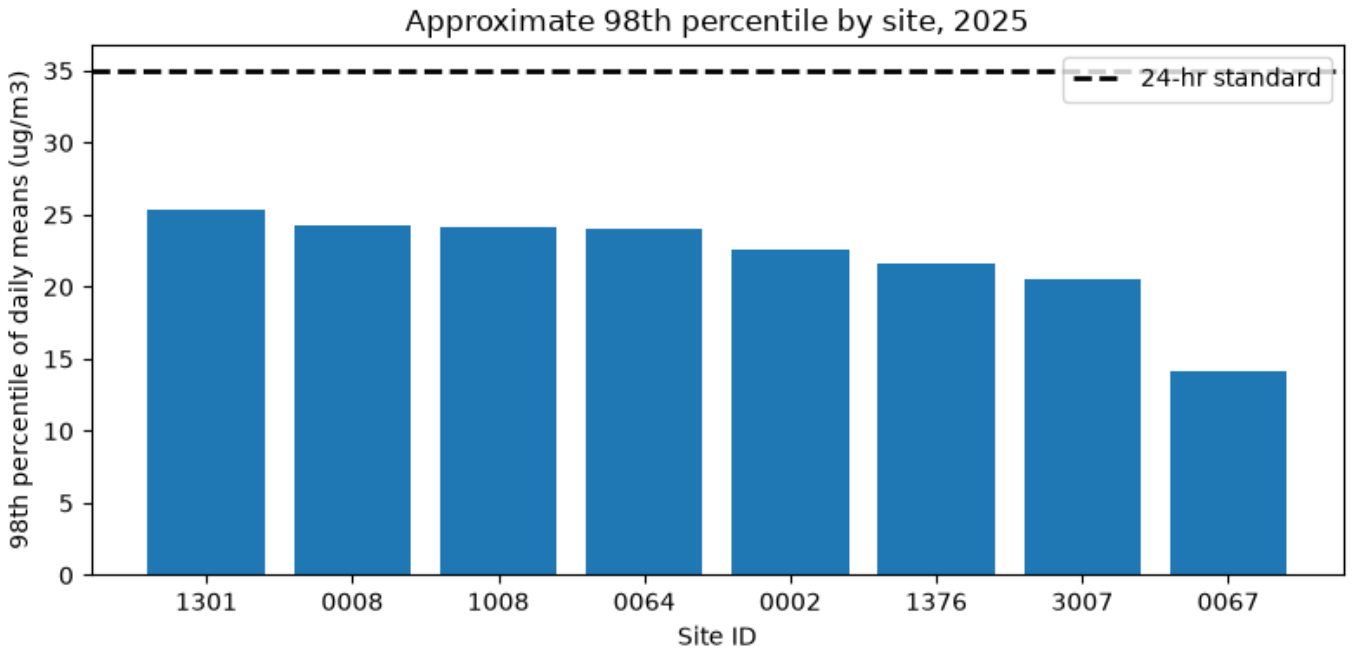
The 24-hour standard targets short, high episodes; the annual standard targets a persistent background. This is **one year** and a simple mean. The official annual value uses quarterly averaging, three years, strict completeness rules, and care about which instruments count.

✍️ ⭐ Stretch (optional, for when you finish early)

Make a bar chart of `site_summary["p98"]` by site with the 24-hour standard as a dashed line (`ax.axhline`). Label the axes and add a title.

```
fig, ax = plt.subplots(figsize=(8, 4), layout="tight")
ax.bar(site_summary.index, site_summary["p98"], color="tab:blue")
ax.axhline(NAAQS_24HR, color="black", linestyle="--", linewidth=2, label="24-hr standard")
ax.set_ylabel("98th percentile of daily means (ug/m3)")
ax.set_xlabel("Site ID")
ax.set_title("Approximate 98th percentile by site, 2025")
ax.legend()
check("bar", len(ax.patches))
plt.show()
```

```
✅ Correct.
```



Step ✓ Wrap-up

Say it in your own words

```
scorecard()   # how did your predictions go?
```

```
You called 1 of 4 predictions.
```

✍️ Last exercise

In one or two sentences, write the most useful thing you learned. Imagine telling a colleague back at the office. (Virtual: paste it in the chat.)

```
my_takeaway = "Averaging time decides the answer: hourly spikes did not exceed the 24-hour
standard."
```

```
from IPython.display import HTML, display
display(HTML(f"<div style='border-left:7px solid #27ae60;padding:12px 18px;border-radius:8px;"
             f"background:rgba(39,174,96,.13);font-size:1.1em'><b>🎓 Your takeaway:</
b><br><i>{my_takeaway}</i></div>"))
```

🎓 Your takeaway:

Averaging time decides the answer: hourly spikes did not exceed the 24-hour standard.

🎯 What you practiced, and what is next

- **Reading a messy file**, fixing it one problem at a time.
- **Checking, not trusting:** units, parameter code, missing data, time zone, and AI-written code.

- **Averaging time and completeness** change the answer. A screening result is not a design value.

Next: **ozone** and the 8-hour average rules, **satellite NO_2** on a map next to these monitors, and a **pipeline** that runs with one command. **Bring one real problem:** what do you do every week at work that looks a bit like this? You will start automating it in the last session.